

WHITEPAPER · V1.0

Cascade

Permissionless prediction markets on real-world DeFi events, resolved with no admin, no oracle, and no dispute window — settled instead by cryptographic proof, via the Attestcoin Protocol.

Networks: Creditcoin Testnet (markets & settlement) · Ethereum Sepolia (source conditions)

Core primitive: Attestcoin cross-chain attestation & Native Query Verifier precompile

Status: Deployed and functionally verified end-to-end on public testnets

Date: September 2026

Abstract

SUMMARY

Cascade is a prediction market protocol for conditions that occur on a different blockchain than the one the market lives on — a whale transfer, a lending pool crossing a borrow-volume threshold, a DEX pool recording another swap. Rather than routing resolution through a centralized oracle, a multisig committee, or a bonded dispute window, Cascade settles markets the moment the underlying event is *cryptographically proven* to have occurred, using the Attestcoin Protocol's cross-chain attestation and Creditcoin's Native Query Verifier precompile. The result is a market whose only trust assumption is the same one users already extend to the blockchains involved: that a sufficiently attested block is final. No party — including Cascade's own operators — can resolve a market against its true on-chain outcome.

1. Problem Statement

Prediction markets that key off real-world or cross-chain events almost universally depend on a trusted party to assert "yes, that happened." In practice this trusted party takes one of three forms: a centralized oracle feed, a multisig of reputed signers, or an optimistic dispute window secured by a bonded challenger. Each of these introduces the same three costs:

- **Delay** — dispute windows and committee sign-off cycles routinely add hours to days before a market can pay out, even when the underlying event was final within seconds of its own chain's finality.
- **Cost** — bonded disputes and oracle infrastructure impose ongoing operational and capital overhead that gets passed to market participants as fees or spread.
- **A manipulation surface** — whoever holds resolution authority can be bribed, censored, coerced, or can simply make an honest mistake. The market's integrity is only as strong as that party's incentives and diligence.

This is particularly wasteful for markets on granular, high-frequency on-chain activity — a specific address's transfer, a lending pool's cumulative borrow volume, a DEX's swap count — because the ground truth for these conditions is *already* sitting in a block on another chain, fully public and independently verifiable. Routing that truth through a human curator or a semi-trusted price/event feed adds a layer of trust the data itself does not require.

2. Solution Overview

Cascade removes the trusted resolver from the critical path entirely. A market's condition is defined declaratively — as a specific event signature emitted by a specific contract on a specific source chain, optionally scoped to one watched address, compared against a numeric threshold. Resolution then proceeds as a three-step cryptographic pipeline rather than a human judgment call:

- 1 **Detect.** An off-chain watcher observes the source chain (Ethereum Sepolia) for a transaction whose logs match an open market's condition.
- 2 **Prove.** Once Creditcoin has attested the block containing that transaction, the Attestcoin Proof Builder service produces a Merkle *inclusion proof* (the transaction is really in that block) and a *continuity proof* (that block is really part of the attested chain history).
- 3 **Verify and settle.** That proof is submitted to `Markets.sol` on Creditcoin, which calls Creditcoin's own Native Query Verifier precompile to independently re-check the proof on-chain — trusting nothing the off-chain resolver claims — before releasing any payout.

If no matching event ever occurs, the market simply expires to NO at its deadline. This fallback path is also permissionless and requires no admin action — there is nothing to disprove, so no proof is required to close it out. The only privileged action anywhere in the protocol is `cancelMarket`, an owner-only escape hatch reserved for a market whose *definition* was malformed at creation — never a lever for altering a market's true outcome.

3. Why Creditcoin and Attestcoin Are Pivotal

Cascade is built specifically around two primitives that, together, make trustless cross-chain resolution practical rather than theoretical.

3.1 Creditcoin's Native Query Verifier Precompile

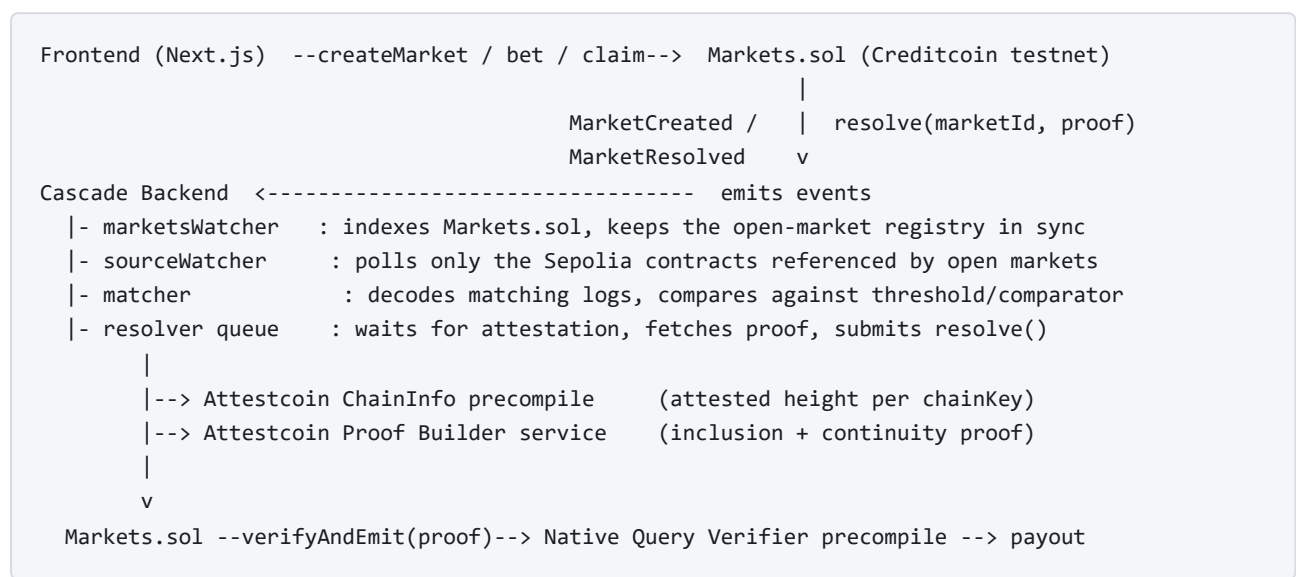
`Markets.sol` lives on Creditcoin testnet, and CTC is staked, won, and paid out there — chosen specifically because Creditcoin exposes the **Native Query Verifier precompile** (`INativeQueryVerifier`) at the protocol level. This precompile lets a smart contract verify, in a single on-chain call, both a foreign-chain transaction's Merkle inclusion proof *and* the continuity of the chain history surrounding it — without the contract trusting anything beyond Creditcoin's own attested state. Implementing equivalent verification in ordinary Solidity, without a native precompile, would be prohibitively expensive in gas and complexity.

3.2 The Attestcoin Protocol

Attestcoin (the `@gluwa/asc-contracts` / `@gluwa/usc-sdk` stack) is the protocol that produces the proofs the precompile verifies. Creditcoin periodically attests to the state of external chains — Sepolia, and eventually Ethereum mainnet and others — under a `chainKey` identifier. Once a source-chain block has been attested, Attestcoin's Proof Builder service can generate a proof that a specific transaction is included in it. `MarketBase.resolve()` hands that proof directly to the verifier precompile via `verifyAndEmit(chainKey, blockHeight, encodedTransaction, merkleProof, continuityProof)`, and only continues into market-resolution logic if the call returns `true`.

3.3 The Combined Guarantee

Neither primitive is sufficient alone. Without the Native Query Verifier precompile, verifying a foreign-chain Merkle proof cheaply on-chain is not practical for a general-purpose smart contract. Without Attestcoin's attestation and proof-building service, there is no trustworthy source for the proof in the first place. Together, they are what makes "permissionless, trustless resolution of a cross-chain condition" achievable at all. Cascade does not reimplement either primitive — it is a market contract and an off-chain indexer built *around* them.

[illegible]

5. Protocol Mechanics

5.1 Market Creation

Anyone may call `createMarket` on Creditcoin, supplying a source contract address, an event signature, an optional watched address, a comparison operator with numeric threshold, and a deadline. The Cascade frontend exposes this as a handful of fixed presets — whale transfer, Aave supply/borrow volume, Aave pool activity, Uniswap swap count — so end users never need to hand-construct raw event signatures. Market creation is intentionally permissionless: safety comes from the condition being independently re-verified at resolution time, not from gatekeeping who may propose a question.

5.2 Betting

Any participant may stake native CTC on the YES or NO outcome via `bet(marketId, outcome)` at any point before the deadline. Stakes accumulate into per-outcome pools; a bettor's eventual share of the winning pool is proportional to their stake within it.

5.3 Watching and Matching

The backend's `sourceWatcher` polls only the Sepolia contracts referenced by currently open markets, keeping off-chain load proportional to active market volume rather than to all of Sepolia. The `marketsWatcher` separately indexes `Markets.sol` events to keep an on-chain market registry current. When a source-chain log matches an open market's `(sourceContract, eventSignature, watchedAddress)` tuple, the `matcher` decodes its value according to the market's `EventTemplate` and compares it against the configured threshold — accumulating the value first for markets of type `Cumulative`, such as running borrow volume.

5.4 Resolution

A matched condition enqueues a resolution job. The resolver worker waits until Creditcoin has attested the relevant Sepolia block, fetches an Attestcoin inclusion-and-continuity proof for the exact matching transaction, and submits `Markets.resolve(...)`. Critically, the contract does not take the resolver's word for it: it re-verifies the proof itself, on-chain, via the Native Query Verifier precompile, before transitioning the market to `Resolved` and decoding the proved transaction's receipt to confirm both that the source transaction succeeded and that its logs genuinely satisfy the market's filter and threshold.

5.5 Expiry (No-Match Fallback)

If a market's deadline passes with no verified match, any party may call `expireMarket(marketId)`, which resolves the market to NO without requiring a proof. Because a NO outcome represents the absence of a matching event, there is nothing that could be proven — the deadline itself is the only evidence needed, and it is already visible on-chain.

5.6 Claiming

Once a market is resolved, winners call `claim(marketId)` to receive `stake + stake × losingPool / winningPool` — their original stake plus a pro-rata share of the losing pool. In the edge case where nobody staked on the winning side, callers instead reclaim their own losing-side stake, ensuring funds are never permanently stranded in the contract.

5.7 Key Function Reference

Function	Contract	Role
<code>createMarket</code>	<code>Markets.sol</code>	Permissionlessly defines a new cross-chain condition, its comparator/threshold, and its deadline; emits <code>MarketCreated</code> .
<code>bet</code>	<code>Markets.sol</code>	Stakes native CTC on YES or NO for an open market before its deadline.
<code>resolve</code>	<code>MarketBase.sol</code>	The proof-verification gate — calls the Native Query Verifier precompile and only proceeds if the cross-chain proof checks out; rejects any query already processed for that market.
<code>_resolveMarket</code>	<code>Markets.sol</code>	Decodes the proved transaction's receipt and logs, confirms the source transaction succeeded, checks the actor filter, and applies the comparator/threshold to finalize (or, for cumulative markets, keep accumulating).
<code>expireMarket</code>	<code>Markets.sol</code>	Permissionless fallback that resolves an unmatched market to NO once its deadline has passed.
<code>claim</code>	<code>Markets.sol</code>	Pays out a winner's stake plus pro-rata share of the losing pool, with a no-winner refund fallback.
<code>cancelMarket</code>	<code>Markets.sol</code>	The protocol's only privileged action — owner-only, reserved for malformed market <i>definitions</i> , never for altering a true outcome.

5.8 Attestcoin Integration Detail

Off-chain, the resolver's `generateProofFor` routine (`backend/src/resolution/proof.ts`) fetches the source transaction's receipt, instantiates an Attestcoin `ProofBuilder` for the relevant `chainKey`, calls `waitUntilHeightAttested` to poll until Creditcoin has attested the containing block (attestation typically trails the source chain by roughly 8–20 minutes), and then calls `getProof(transactionHash)` to retrieve the inclusion and continuity proof. The resolution queue then submits this proof via `Markets.resolve(...)`, computing an appropriate gas limit from the continuity proof's root count, and parses the resulting `MarketResolved` event to confirm the on-chain outcome. Proofs are cached per (`chainKey`, `txHash`), so a single source transaction that happens to satisfy several open markets costs only one Proof Builder round trip.

6. Security Model

Cascade's security rests on minimizing the number of parties whose honesty a bettor must assume, and pushing every remaining assumption down to primitives that are already load-bearing for the chains involved.

What a bettor must trust

- Creditcoin's own consensus and the correctness of its Native Query Verifier precompile.
- Attestcoin's attestation of Sepolia block headers into Creditcoin's `ChainInfo` state.
- Standard smart-contract assumptions: `Markets.sol`'s own logic is correctly implemented and immutable post-deployment.

What a bettor does *not* need to trust

- Cascade's own backend or resolver operator — it can only submit proofs, never fabricate outcomes; the contract independently re-verifies every proof.
- Any oracle committee, multisig, or bonded challenger — none exists in the protocol.
- Market creators — a malicious or careless market definition cannot force a false outcome, since resolution logic re-derives the outcome from the proved transaction itself.

The `resolve` function additionally guards against proof replay by tracking `processedQueries[marketId][queryId]`, and against post-deadline resolution races by checking `block.timestamp < deadline` before finalizing. The `claim` function is reentrancy-guarded and includes an explicit no-winner refund path so that funds cannot become permanently stuck if a market resolves with an empty winning pool.

7. Implementation and Technology Stack

Layer	Technology	Purpose
Smart contracts	Solidity 0.8.28, Hardhat 2, OpenZeppelin	Markets.sol / MarketBase.sol — market lifecycle and proof-gated resolution
Cross-chain verification	@gluwa/asc-contracts , @gluwa/usc-sdk	Attestcoin proof generation and INativeQueryVerifier precompile bindings
Backend	Node.js, TypeScript, Express, viem, ethers v6	Two-chain indexer, matcher, resolution job queue, REST API
Persistence	Firebase Admin / Firestore	Frontend-authored market metadata (question text, presets) not derivable from chain state alone
Frontend	Next.js 16, React 19, wagmi, viem, RainbowKit	Market creation UI, betting flow, live on-chain reads
Source chain	Ethereum Sepolia	Where the real DeFi conditions occur — ERC-20 transfers, Aave Supply/Borrow, Uniswap V3 swaps
Markets chain	Creditcoin Testnet (chain ID 102031)	Hosts Markets.sol and the Native Query Verifier precompile; settlement currency is CTC

7.1 Deployed Contracts

Contract	Chain	Address	Notes
Markets	Creditcoin Testnet (102031)	0x0a0d9f5875b54d9adfa4Fc121D9C5f70EEE2450f	Single contract for every market — no per-market deploys
TestToken (CTT)	Sepolia (11155111)	0xE8473Df91c4EB7cf0972E9dfb856329B84154920	Test ERC-20 used as a source-chain condition for the end-to-end run

8. End-to-End Testnet Validation

The full pipeline — create, bet, source event, detect, prove, resolve, claim — was executed against live public testnets on 10 September 2026. Market 1 was configured as a `SingleEvent` / `SingleWordValue` market watching CTT `Transfer` events from a specific Sepolia address, resolving YES if the transferred value was greater than or equal to 500 CTT.

Step	Chain	Transaction	Purpose
<code>TestToken</code> deploy	Sepolia	<code>0x07fd76fb0c3b8387f3e95308ea46afdf99d9088615d239171b3e33e11b15896b</code>	Deploy the CTT test ERC-20 used as the market's source contract
<code>createMarket</code>	Creditcoin Testnet	<code>0xc8d8e039dfeed84795305097410450fc3572e5c000353d327a2667154f5bde5c</code>	Create Market 1 — watch CTT <code>Transfer</code> , threshold 500e18, GTE
<code>bet</code> (YES)	Creditcoin Testnet	<code>0x54533ca318e7195a09b16368e3ffd782a58bac08775d1f2af3df585ca5b8e78e</code>	Stake 0.002 CTC on YES
<code>bet</code> (NO)	Creditcoin Testnet	<code>0xd6d8e7aeb314a87c85639db780ffaface53d09ebb9f9d66d74297d7665c121752</code>	Stake 0.001 CTC on NO
Source transfer	Sepolia	<code>0x52cef4fcac54534847a3b44a06bd572a8af31b258967c38e9ca962700c32a67f</code>	600 CTT transfer matched by the watcher; this attempt exhausted retries on flaky public RPC timeouts
Source transfer (retry)	Sepolia	<code>0x2a16329f3635742b8d106dc0b9d989382a8a6f73ae8dc8a91d5ebbd8ae45e7d2</code>	750 CTT transfer — the transaction actually proven and resolved against
<code>resolve</code>	Creditcoin Testnet	<code>0xf4a474c2538dddc565607be7cd9c9c2afd3c6d2ccf20e5ec254ec94f869dcf1</code>	Attestcoin proof for the 750 CTT transfer verified on-chain; emits <code>MarketResolved(1, true)</code>
<code>claim</code>	Creditcoin Testnet	<code>0x2ef98c62589d1d29b604085bb1800319cc3ccd20ccbb4cae7d2dab191e0d830f</code>	Payout — 0.002 CTC stake + 0.001 CTC pro-rata winnings = 0.003 CTC

Observation. The one failed resolution attempt in this run was caused by public Sepolia RPC timeouts during proof retrieval — an infrastructure reliability concern, not a protocol or cryptographic failure. The retry against the second, larger transfer resolved and paid out correctly, demonstrating the full detect → attest → prove → verify → settle → claim pipeline end-to-end on public infrastructure.

9. Future Work

- **Additional source chains.** Extending beyond Sepolia to further chains Attestcoin attests, widening the space of provable DeFi conditions Cascade markets can reference.
- **Richer event templates.** Support for multi-leg conditions (e.g., conjunctions across two events) beyond the current single-event and cumulative-value templates.
- **Resolver decentralization.** Opening the resolver role to any party willing to submit a valid proof and earn a small fee, removing reliance on a single operated resolver process for liveness (the contract's trust model already permits this — only liveness is currently centralized).
- **Mainnet deployment.** Migrating from Creditcoin and Sepolia testnets to their production counterparts once Attestcoin attestation is available there.
- **Market discovery and liquidity tooling.** Improved UI for browsing, filtering, and eventually providing liquidity across many concurrently open markets.

10. Conclusion

Cascade demonstrates that prediction markets on cross-chain DeFi activity do not need to trade decentralization for resolvability. By treating market resolution as a cryptographic verification problem — rather than a governance or oracle problem — and by building directly on Creditcoin's Native Query Verifier precompile and the Attestcoin Protocol's attestation infrastructure, Cascade settles markets the moment their underlying condition is provably true on the source chain, with no admin, no oracle, and no dispute window standing between an event and a payout. The end-to-end testnet run described in Section 8 confirms this pipeline functions correctly against live, independent public chains — from market creation, through real Sepolia activity, to a cryptographically verified, on-chain payout.